Multi-class Semantic Segmentation on the PanNuke Histological Dataset

Table of contents

Table of contents---------- 2
Introduction----------4
Problem description---------- 4
PanNuke Dataset----------4
Data Preparation---------- 6
Loss Function & Metrics---------- 6
Loss Function----------6
Evaluation Metric---------- 6
Data Augmentation----------6
Models----------7
Model 1 — SimpleUNet (Baseline)----------7
Model 2 — SimpleUNet2 (Dropout + Augmentation)---------- 7
Model 3 — UNet++---------- 7
Model 4 — Pretrained UNet++ (ResNet34 encoder)---------- 8
Results----------9
Model 1 - SimpleUnet---------- 9
SimpleUNet predictions---------- 11
Model 2 - SimpleUNet2----------12
SimpleUNet2 predictions----------13
Model 3 - UNet++---------- 15
UNet++ Predictions---------- 16
Model 4 - Pretrained UNet++---------- 18
Overall result - Conclusion---------- 21
Requirements---------- 23
Libraries----------23
References:---------- 23

Introduction

Problem description

The project, “Multi-class Semantic Segmentation on the PanNuke Histological Dataset”, as the title describes addresses the task of multi-class semantic segmentation on histological images from the **PanNuke dataset**.

The PanNuke dataset is a Nuclei Instance Segmentation and Classification type dataset and it can be downloaded from the following page: https://huggingface.co/datasets/RationAI/PanNuke.

The goal was to classify every pixel in a 256×256 tissue image into one of six categories: Background, Neoplastic, Inflammatory, Connective, Dead, and Epithelial cells.

The semantic segmentation of histological data is a challenging task due to:

- Small cell sizes relative to the image
- Visual similarity between cell types
- Staining intensity varies significantly between different tissue samples, making it harder for the model to generalize

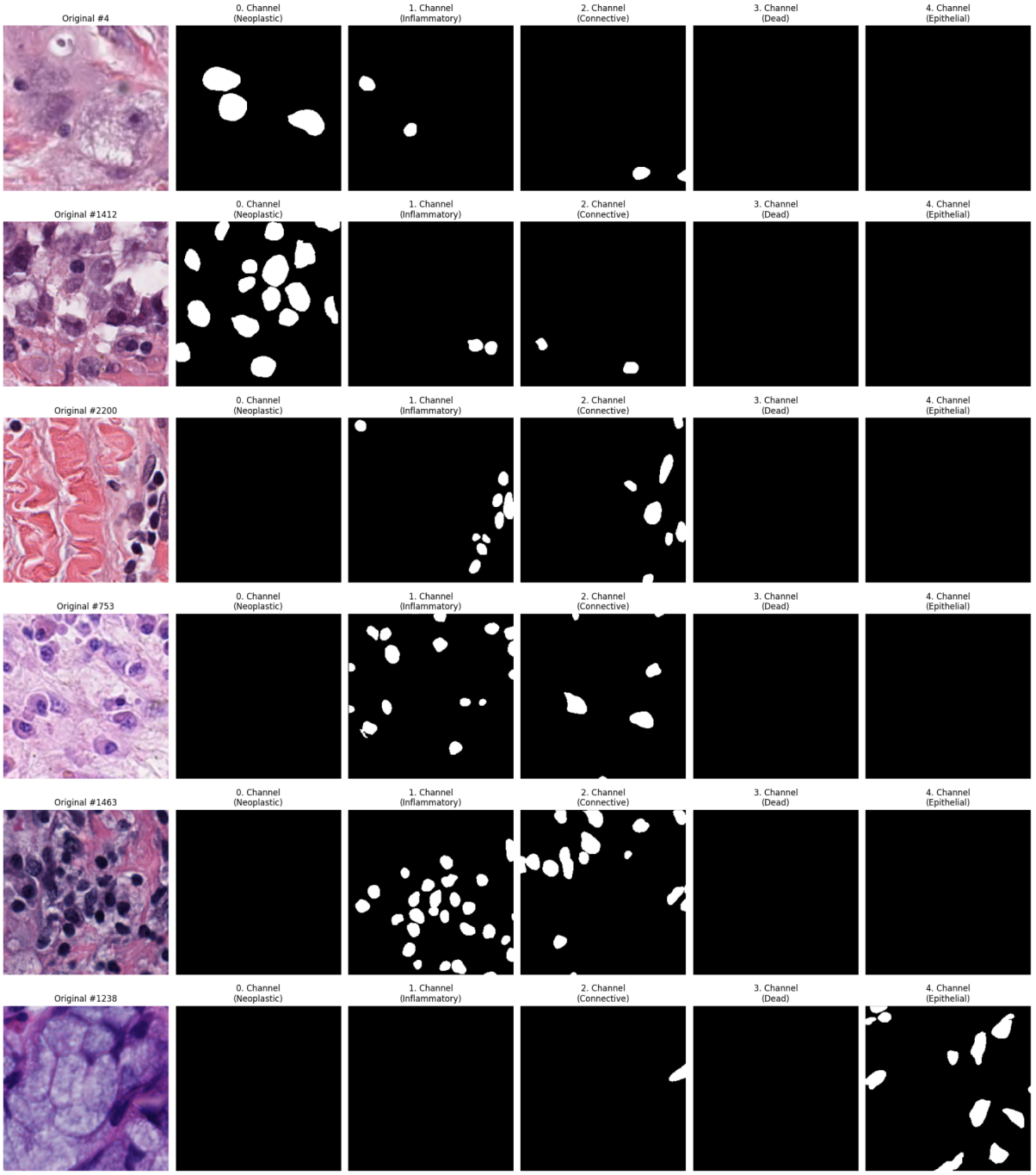
In the project four progressively more complex models have been trained, optimized and compared, starting from a baseline model and ending with a pretrained one.

PanNuke Dataset

PanNuke is a publicly available histological dataset for nucleus instance segmentation and classification. Three Folds of dataset can be downloaded. In this project, only Fold1 was used, which contained 2656 image-mask pairs.

Index	***Class name***	***Description***
0	Background	No cell present
1	Neoplastic	Tumor cells
2	Inflammatory	Immune cells
3	Connective	Connective tissue cells
4	Dead	Necrotic cells
5	Epithelial	Epithelial cells

Note: In the original dataset the cell instances were the following: 0: Neoplastic; 1: Inflammatory; 2: Connective; 3: Dead; 4: Epithelial; 6: Background. The values were changed based on the above table in the project.



Data Preparation

The original 6-channel instance masks were converted into a single semantic label map, where each pixel is assigned one class label (0–5). This format is directly compatible with the Dice Loss and the model's output.

The dataset was split into:

- **Train:** 70%
- **Validation:** 15%
- **Test:** 15%

Loss Function & Metrics

Loss Function

All models were trained using **Dice Loss** — a soft, differentiable loss that directly optimizes pixel overlap between prediction and ground truth:

Dice Loss = 1 - (2 × intersection) / (union)

Evaluation Metric

Model performance was measured using **Mean Dice Score** — the average Dice coefficient computed independently for each class:

Mean Dice = average of Dice scores across all 6 classes

Data Augmentation

To improve generalization, the following augmentations were applied randomly during training:

Augmentation	Probability
Horizontal flip	50%
Vertical flip	50%
Random rotation (90°/180°/270°)	50%
Brightness adjustment (±20%)	35%
Contrast adjustment (±20%)	35%
Saturation adjustment (±20%)	35%

Models

Model 1 — SimpleUNet (Baseline)

A standard U-Net with 3 encoder stages and 3 decoder stages connected by skip connections. No dropout, no augmentation.

Configuration:

- Batch size: 64
- Learning rate: 0.001
- Epochs: 60
- Optimizer: Adam

Model 2 — SimpleUNet2 (Dropout + Augmentation)

Same architecture as SimpleUNet but with Dropout2d(0.2) added to each decoder block, combined with data augmentation.

Changes from SimpleUNet:

- Dropout (p=0.2) added to all decoder blocks
- Data augmentation enabled
- Learning rate decreased: 0.001 → 0.0005

Configuration:

- Batch size: 64
- Learning rate: 0.0005
- Epochs: 60
- Optimizer: Adam

Model 3 — UNet++

A more complex architecture that replaces direct skip connections with a grid of intermediate refinement nodes.

Configuration:

- Batch size: 16
- Learning rate: 0.0001
- Epochs: 300
- Optimizer: Adam

Changes from SimpleUNet2:

	SimpleUNet2	**UNet++**
Encoder depth	3 stages	4 stages
Skip connections	Direct (one jump)	Dense (through intermediate nodes)
Dropout	Decoder only	All 6 intermediate nodes
Learning rate	0.0005	0.0001
Batch size	64	16

Model 4 — Pretrained UNet++ (ResNet34 encoder)

UNet++ architecture with a **ResNet34 encoder pretrained on ImageNet**, implemented via segmentation-models-pytorch.

Two-part architecture:

- **Encoder:** ResNet34 pretrained on 1.2M ImageNet images — already knows edges, textures, and shapes
- **Decoder:** UNet++ trained from scratch on PanNuke

Configuration:

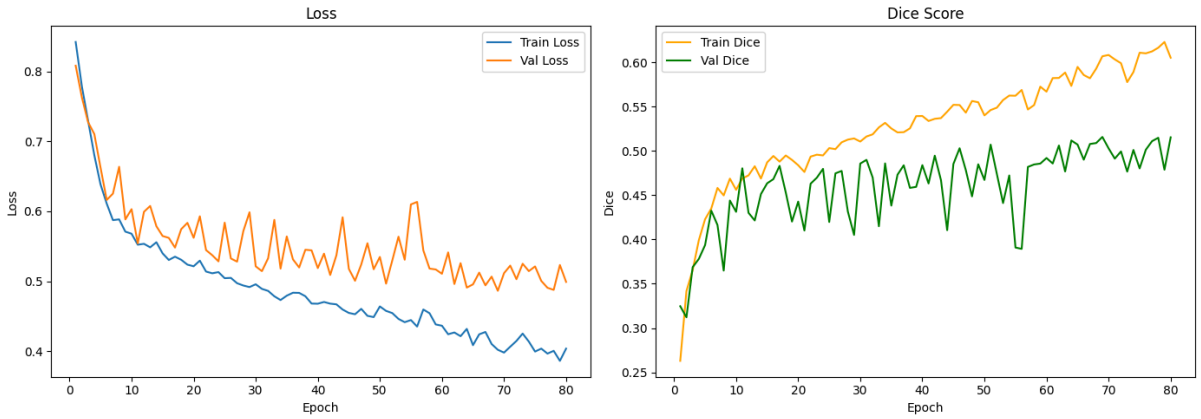
- Batch size: 16
- Learning rate: 0.0001
- Epochs: 200
- Optimizer: AdamW (weight_decay=1e-4)

Results

In this project, four models were trained and evaluated on the PanNuke histological dataset. For each model, the training and validation loss as well as the training and validation Dice scores were monitored throughout the training process. At each epoch, if the model achieved a higher validation Dice score than all previous epochs, the model checkpoint was saved .

After training, the saved checkpoints were loaded and evaluated on the held-out test set, which the models had never seen during training. A set of test samples was randomly selected and visualized, showing the original image, the ground truth mask, and the model's prediction side by side.

Model 1 - SimpleUnet



After finishing the training on our SimpleUNet model we can see the followings:

Loss graph:

- Train Loss decreases smoothly and steadily
- Val Loss decreases but fluctuates a lot
- After the 30th epoch the gap between the two loss curves starts to increase, which is a sign of the start of overfitting

---

Dice graph:

- Train Dice increases nicely
- Val Dice increases but fluctuates a lot - we have to take into consideration that we calculate a mean Dice score for all 6 classes
- After the 30th epoch the gap between the two dice curves starts to increase, which is a sign of the start of overfitting

---

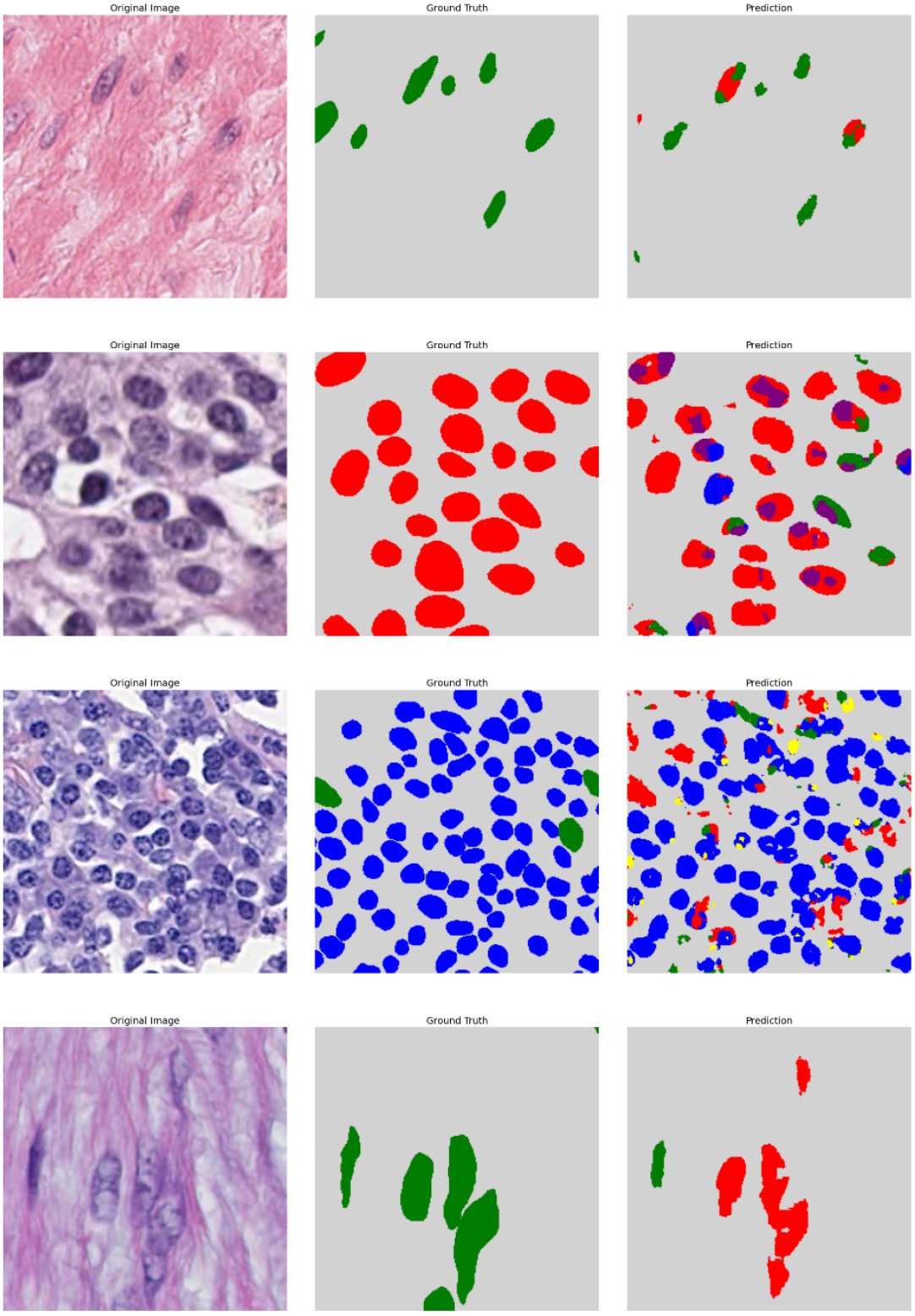
With the above configuration, the SimpleUNet model reached its best result without overfitting at **Epoch [31/80]**, where the results were the followings:

- [ ] Train Loss: 0.4892 | Train Dice: 0.5162 ||| Val Loss: 0.5145 | Val Dice: 0.4898

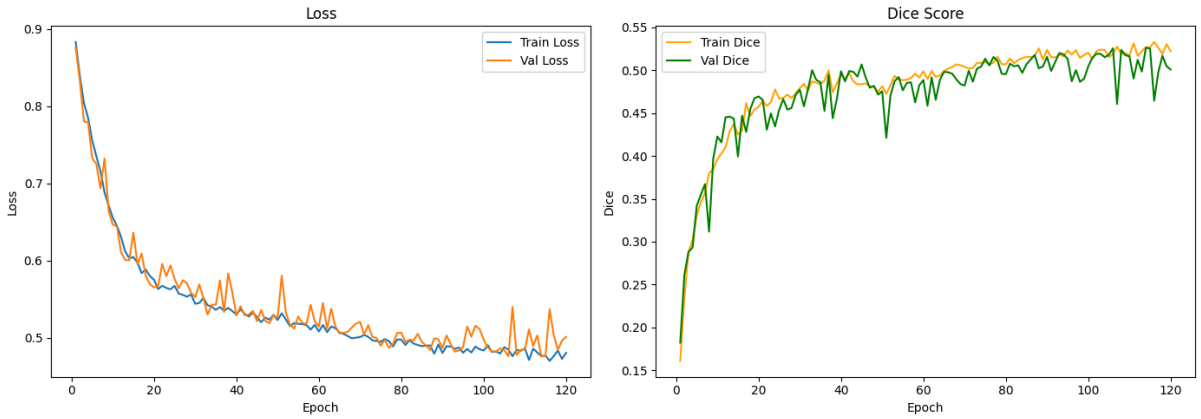
After this point the Train-Val Dice gap widens, while the Val Dice gains only slightly: the highest Val Dice of the run (0.5157 at Epoch [69/80]) is therefore mostly a sign of overfitting and noise, not a real improvement.

While some improvements could be made by adjusting the hyperparameters, the model's simple architecture limits its ability to achieve more significant results. Therefore, more complex architecture and data augmentation techniques were followed.

SimpleUNet predictions



Model 2 - SimpleUNet2



After finishing the training on our SimpleUNet2 model we can see the following:

Loss graph:

- Train Loss decreases smoothly and steadily
- Val Loss decreases with only small fluctuations
- The decrease slows down after about the 90th epoch, but does not stop

---

Dice graph:

- Train Dice increases nicely
- Val Dice increases nicely with some fluctuations - we have to take into consideration that we calculate a mean Dice score for all 6 classes
- The increase slows down after about the 90th epoch (Val Dice around 0.51 - 0.52), but does not stop

---

- => model generalizes well
- => no overfitting
- => the gap between Train and Val Dice stays small (mostly below 0.03) for all 120 epochs

---

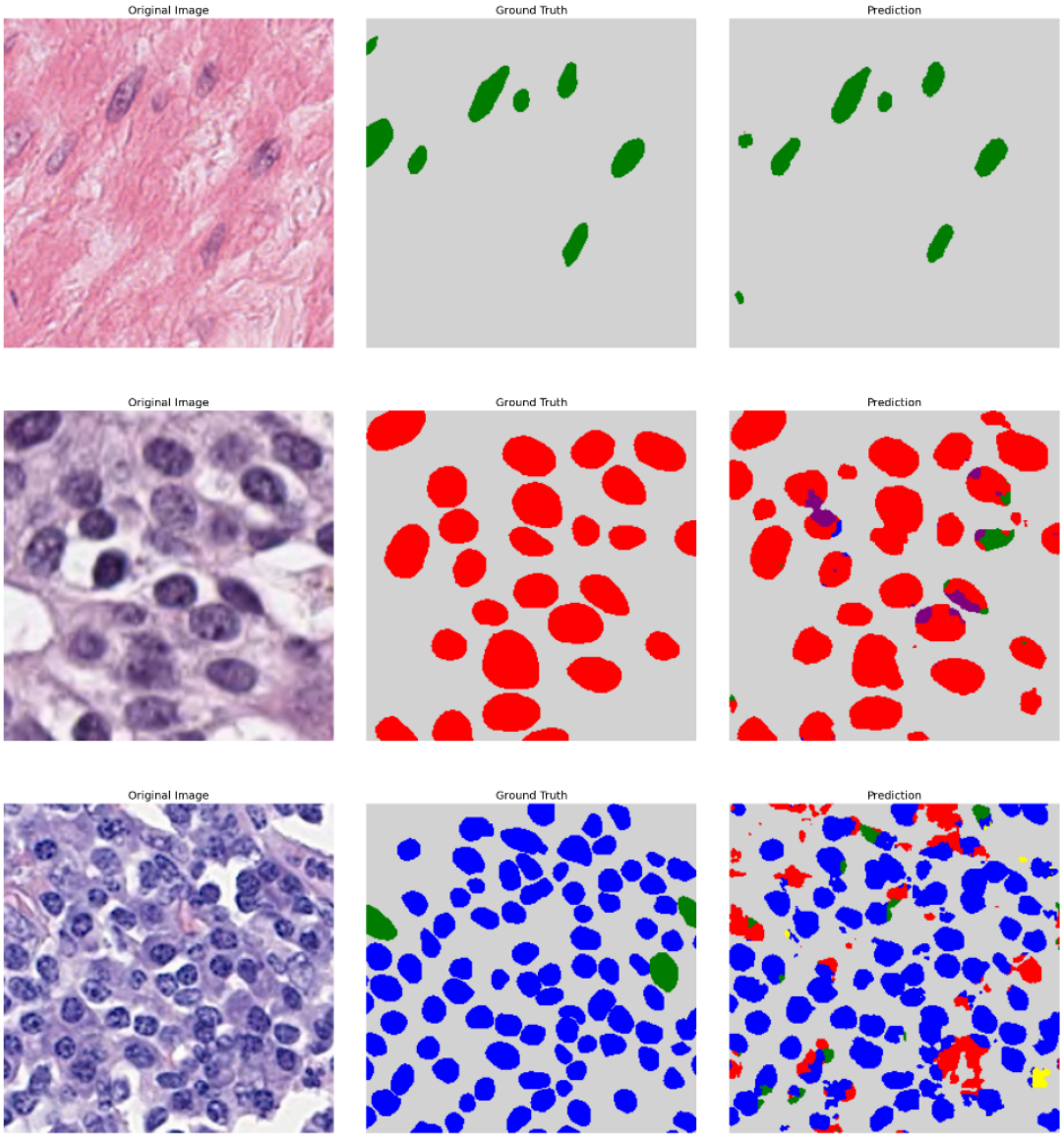
With the above configuration, the SimpleUNet2 model could reach its best at **Epoch [114/120]**, where the results were the following:

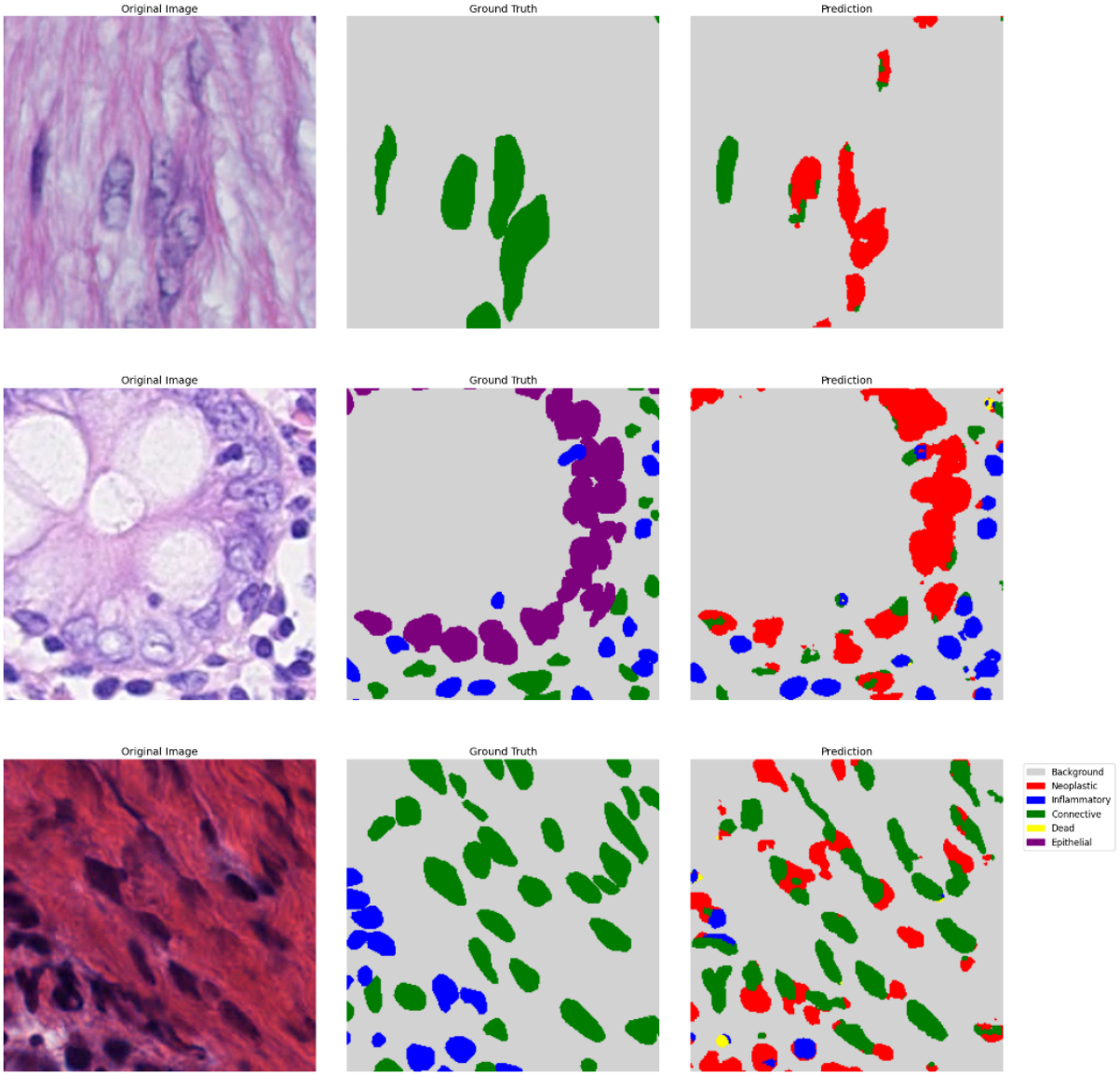
☐ Train Loss: 0.4764 | Train Dice: 0.5267 ||| Val Loss: 0.4757 | Val Dice: 0.5262

Comparing SimpleUNet with SimpleUNet2, dropout layers were introduced into the model, the learning rate was decreased and data augmentation techniques were applied. These modifications led to a significant improvement in training — the learning process became more stable and efficient.The

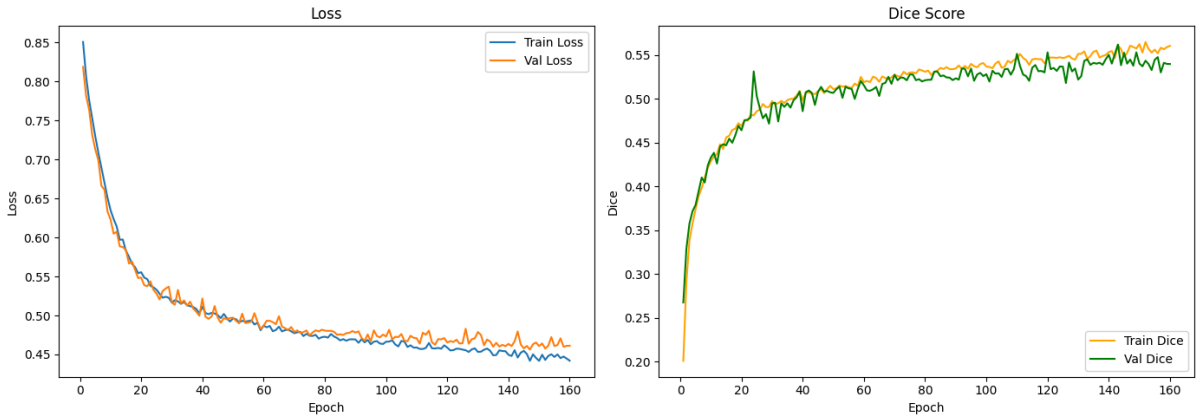
small gap between Train and Val Dice indicates that the overfitting problem from SimpleUNet was successfully addressed through dropout and data augmentation. Looking at the results, the model reached its best Val Dice (0.5262) at epoch 114 with practically no gap between Train and Val Dice. To increase the chances of achieving a higher Dice score, faster, a more complex architecture will be introduced in the next step.

SimpleUNet2 predictions




Original Image
Ground Truth
Prediction
Original Image
Ground Truth
Prediction
Original Image
Ground Truth
Prediction
Background
Neoplastic
Inflammatory
Connective
Dead
Epithelial

Model 3 - UNet++



After finishing the training of the UNetPlusPLus model we can see the following:

Loss graph:

- Train and Val Loss both decrease smoothly
- Around epoch 150-160 the gap between the 2 losses starts to increase slowly

---

Dice graph:

- Train Dice increases nicely
- After epoch 50 the improvement slows down significantly
- Around epoch 150-160 the gap between the 2 dicess starts to increase slowly

---

With the above configuration, the UNet++ model could reach its best at **Epoch [143/160]**, where the results were the following:

☐ Train Loss: 0.4460 | Train Dice: 0.5610 ||| Val Loss: 0.4795 | Val Dice: 0.5620

At Epoch 143:

- => model generalizes well
- => no overfitting at this epoch
- => this Val Dice is a peak above the usual 0.54 - 0.55 range of the last epochs, the learning slows down but does not entirely stop, however a slight sign of overfitting is there (the Train-Val gap slowly grows towards the end)

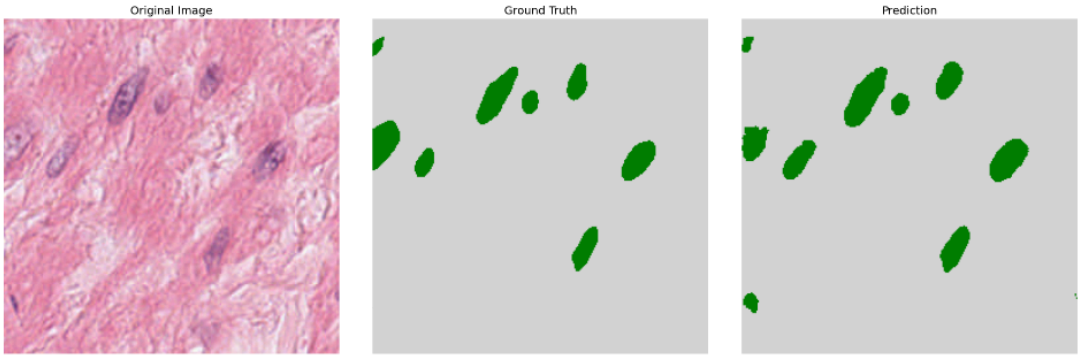
Comparing SimpleUNet2 with Unet++, the following are the differences:

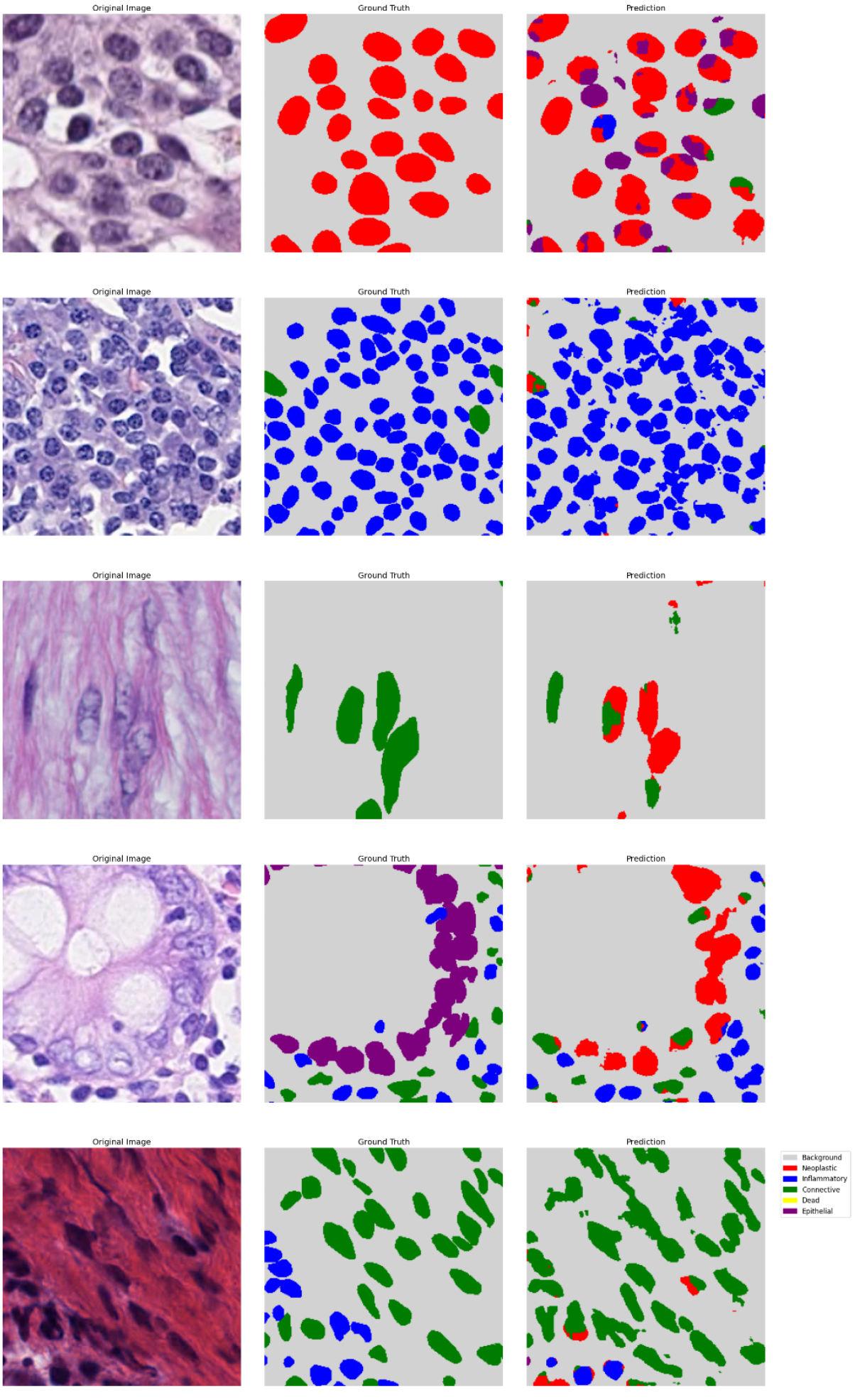
1. Deeper architecture:
   - SimpleUNet2: 3 encoder + 3 decoder blocks
   - UNet++: 4 encoder blocks + multiple refinement columns
2. Intermediate nodes:
   - SimpleUNet2: encoder connects directly to decoder (one jump)
   - UNet++: encoder connects through intermediate nodes that gradually refine the features
3. More Dropout:
   - SimpleUNet2: dropout only in decoder
   - UNet++: dropout in all intermediate nodes

Comparing SimpleUNet2 with UNet++, the more complex architecture brought an improvement — Val Dice increased from 0.5262 to 0.5620. However, SimpleUNet2 was trained only for 120 epochs (UNet++ for 160) and its learning had slowed down but not stopped, so we can not be entirely sure that further training would not have brought it closer to, or even above, the UNet++ result.

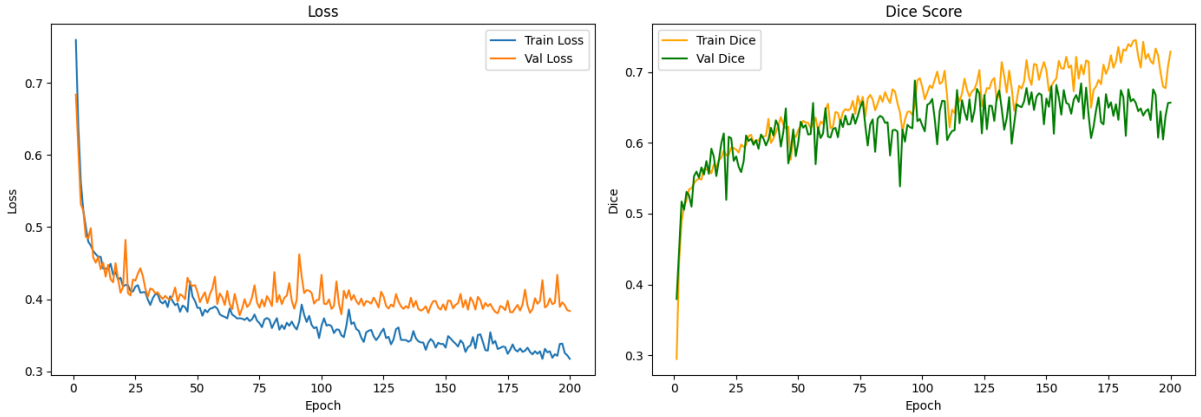
The learning rate was decreased from 0.0005 to 0.0001 and the batch-size was also decreased from 64 to 16. The dense skip connections and intermediate refinement nodes allowed the model to learn more detailed features, assuming in better segmentation performance. However, a slight sign of overfitting appears towards the end of the training: the Train-Val Dice gap slowly grows from about 0.01 (epochs 80 - 125) to about 0.02 (epochs 148 - 160), while the Val Dice has not started to decrease. This suggests that further regularization or a pretrained encoder could push the results even higher.

UNet++ Predictions




Original Image
Ground Truth
Prediction
Original Image
Ground Truth
Prediction
Original Image
Ground Truth
Prediction
Original Image
Ground Truth
Prediction
Original Image
Ground Truth
Prediction
Background
Neoplastic
Inflammatory
Connective
Dead
Epithelial

Model 4 - Pretrained UNet++



After finishing the training of the pretrained UNet++ model we can see the following:

Loss graph:

- Train Loss decreases with small fluctuations (from ~0.76 to ~0.32)
- Val Loss drops quickly in the first ~20 epochs (from ~0.68 to ~0.41), then it hardly improves (around 0.39 after epoch 60) and fluctuates more than Train Loss
- A growing gap between Train and Val Loss is visible from epoch 50 onwards, indicating overfitting as the model continues to train

---

Dice graph:

- Both Train and Val Dice increase rapidly in the first 25 epochs - as expected (Val Dice is ~0.55 already at epoch 8)
- After epoch 50 the improvement slows down significantly
- A consistent increasing gap between Train Dice and Val Dice is visible: it grows from about 0.01 (epochs 20 - 60) to about 0.04 (epochs 140 - 160) and about 0.07 (epochs 180 - 200), where Train Dice is ~0.72 and Val Dice ~0.65, indicating moderate overfitting as continuing to train the model.

With the above configuration, the pretrained UNet++ model could reach its best result without overfitting at **Epoch [76/200]**, where the results were the following:

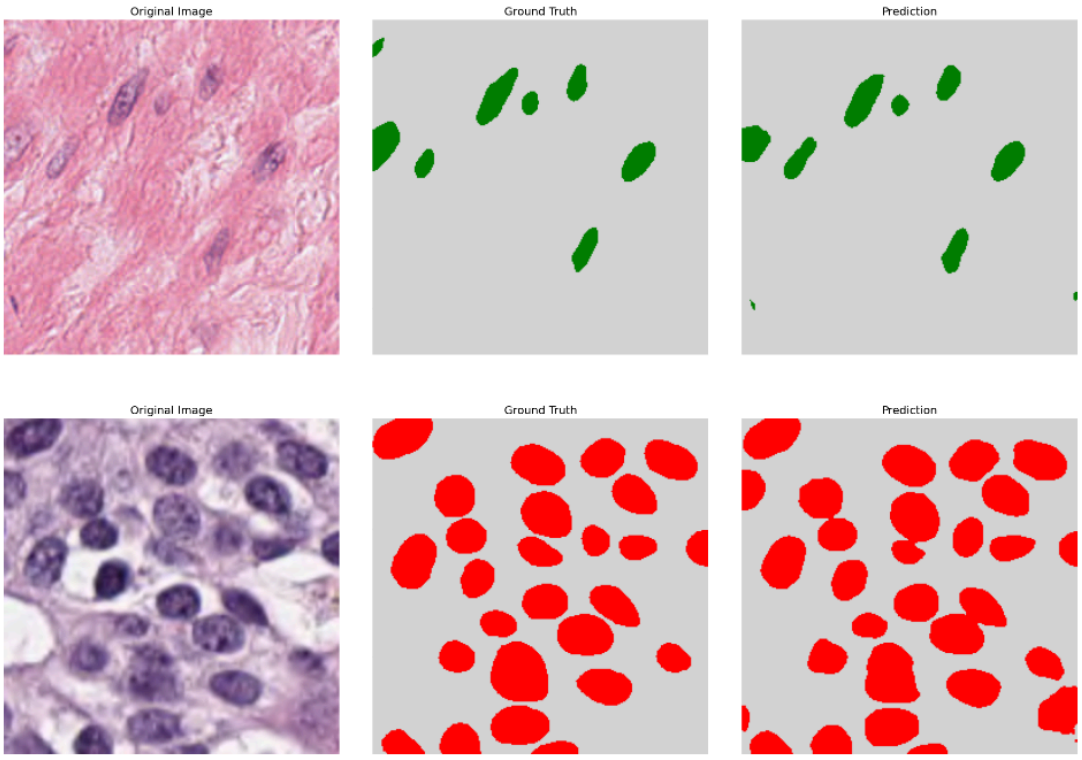
☐ Train Loss: 0.3612 | Train Dice: 0.6649 ||| Val Loss: 0.3995 | Val Dice: 0.6584

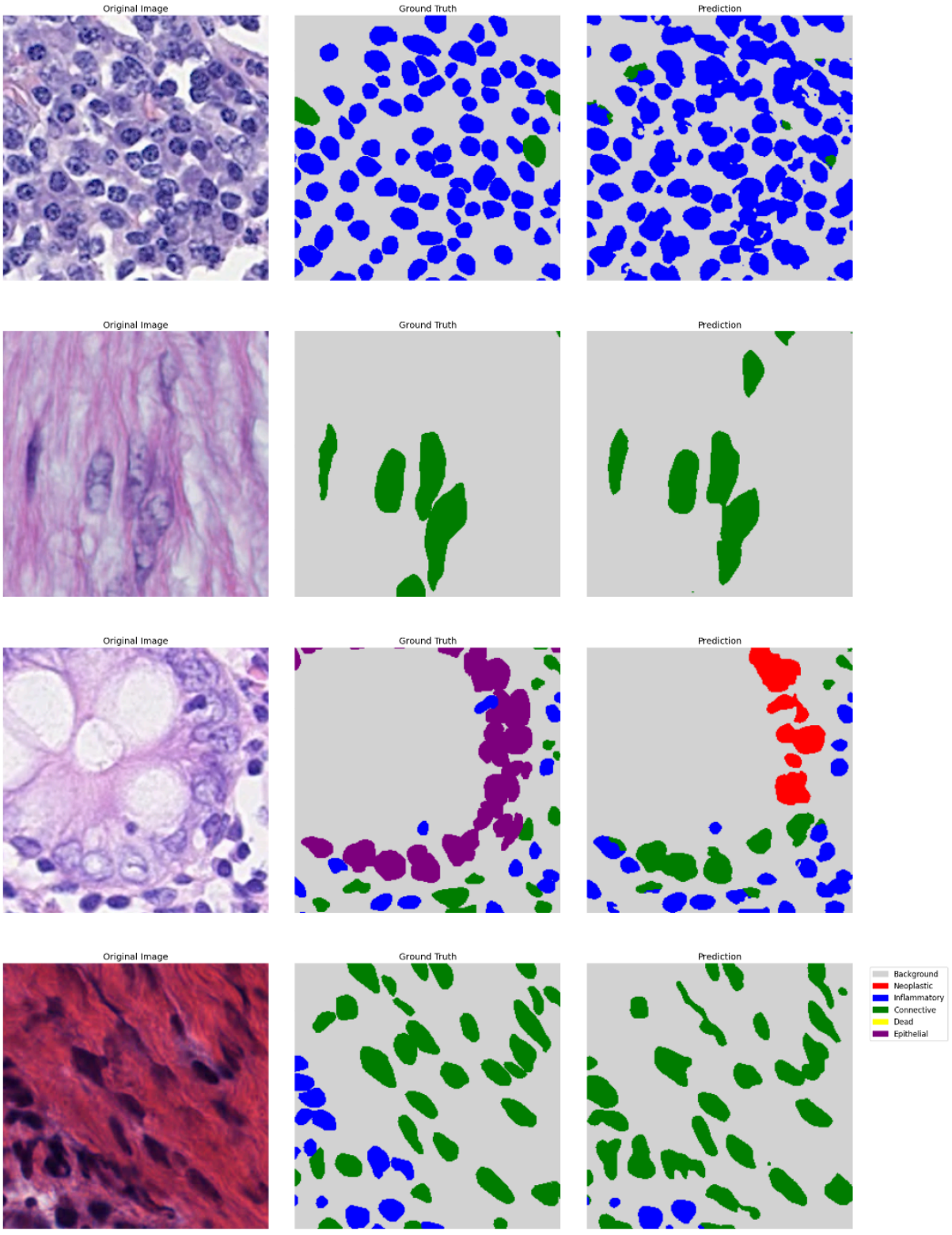
At Epoch 76:

- => no significant overfitting (the Train-Val Dice gap is only ~0.01, it grows clearly after epoch ~80)
- => the Val Dice fluctuates from epoch to epoch, but this value is the top of a fairly stable stretch , not an isolated spike
- => later epochs reach a higher Val Dice (up to 0.6878), but only as single peaks with a growing Train-Val gap

Comparing UNet++ with pretrained UNet++, the introduction of a pretrained ResNet34 encoder brought a clear improvement — Val Dice increased from 0.5620 (epoch 143) to 0.6584 (epoch 76), and even the typical Val Dice of the later epochs (~0.65) is clearly above the ~0.54 - 0.55 of UNet++. The pretrained encoder already knows edges, textures, and shapes from 1.2 million ImageNet images, allowing the model to converge much faster and reach higher Dice scores. Because the overfitting is more visible here than in the previous models, further regularization techniques such as stronger dropout or additional augmentation could help and push the results even higher.

Predictions Pretrained UNet++




Original Image
Ground Truth
Prediction
Original Image
Ground Truth
Prediction
Original Image
Ground Truth
Prediction
Original Image
Ground Truth
Prediction
Background
Neoplastic
Inflammatory
Connective
Dead
Epithelial

Overall result - Conclusion

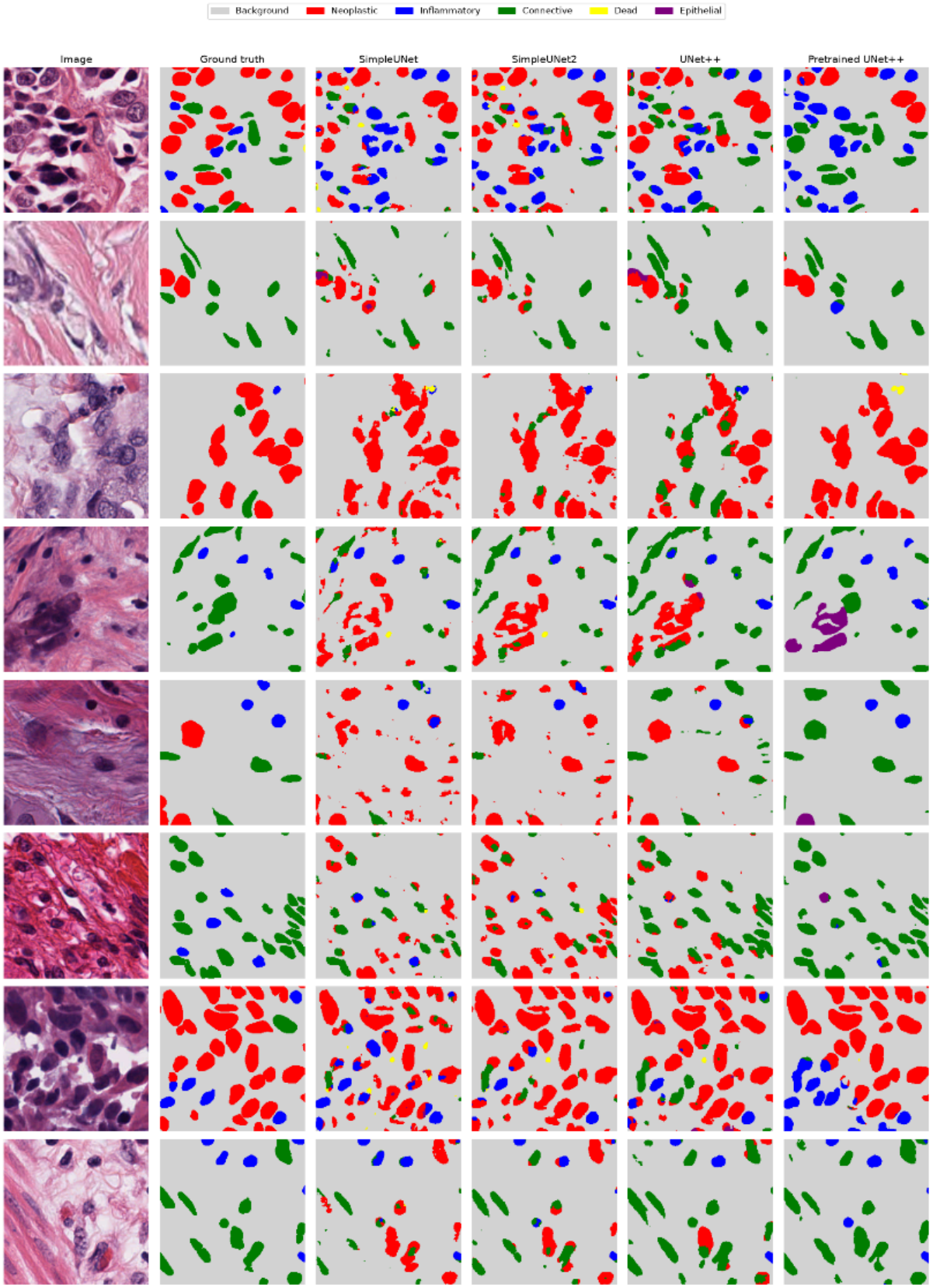
Model	Best Val Dice	Best Epoch	Epochs trained	Test Dice (per image)	Test Dice (per batch)
SimpleUNet	0.4898	31	80	0.4159	0.4926
SimpleUNet 2	0.5262	114	120	0.4551	0.5311
UNet++	0.5620	143	160	0.4806	0.5751
Pretrained UNet++	**0.6584**	76	200	**0.6266**	**0.6488**

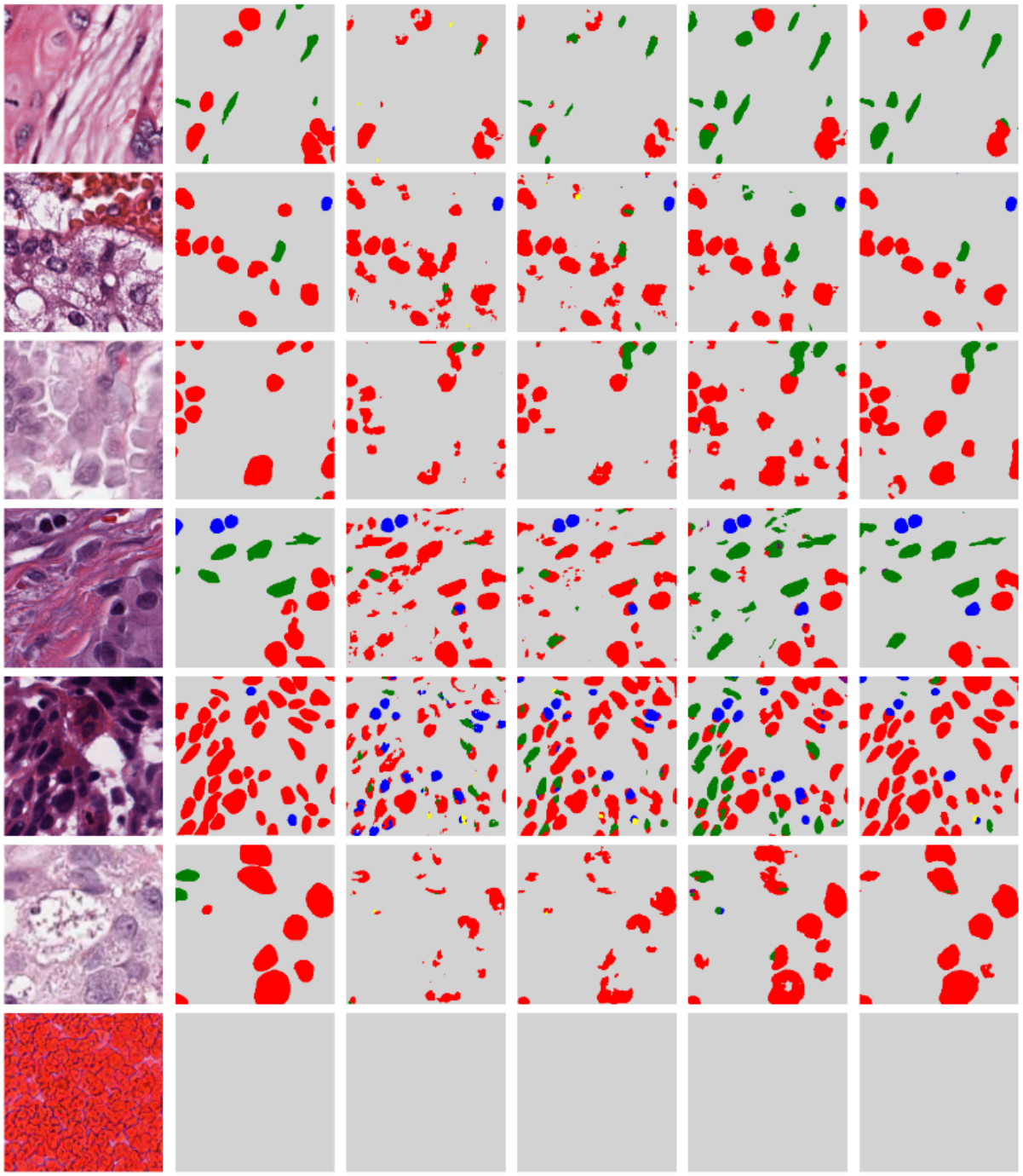
Per image: Dice computed for each of the 398 test images and averaged (05_DataEvaluation_summary.ipynb). Per batch: the value printed at the end of notebooks 01–04, averaged over batches. Both metrics give the same ranking.

The project demonstrated a progressive improvement in semantic segmentation performance on the PanNuke histological dataset through four stages. The models demonstrate a reasonable ability to identify the dominant cell classes in some cases, such as Neoplastic (red) and Connective (green), with generally correct shapes and locations. In several cases the predictions closely match the ground truth masks. However, the models struggle with smaller and rarer cell types — Inflammatory (blue) and Dead (yellow) cells are frequently missed or misclassified. In some cases class confusion occurs, where the model predicts the wrong cell type for a region.

1. **SimpleUNet** - simple baseline model
2. **SimpleUNet2 -** improved performance through dropout and data augmentation + hyperparameter modification
3. **UNet++ -** improved performance through dense skip connections and intermediate refinement nodes - architecture improvement + hyperparameter tuning (data augmentation was applied)
4. **Pretrained UNet++** achieved the best result (Val Dice: 0.6026) by leveraging a ResNet34 encoder pretrained on ImageNet (data augmentation was applied)

Model Predictions Comparison


Background
Neoplastic
Inflammatory
Connective
Dead
Epithelial
Image
Ground truth
SimpleUNet
SimpleUNet2
UNet++
Pretrained UNet++



Future work and limitations:

- Apply stronger regularization or hyperparameter tuning to reduce the loss function fluctuation in the pretrained model
- Explore additional augmentation techniques or adding the Fold2 and Fold3 to expand the dataset - In total there are more than 7 thousand training patches within all of the three folds
- Experiment with a more advanced architecture: examples - Attention UNet architecture, HoVer-Net, CellViT. Based on the literature in some cases they managed to achieve a 0.80 - 0.88 precision using the HoVer-Net and CellViT models.

- Predicting the tissue type of the image - classification problem. Each image belongs to a tissue type from which the sample originates, belonging to one of the tissue categories:
    - Adrenal Gland
    - Bile Duct
    - Bladder
    - Breast
    - Cervix
    - Colon
    - Esophagus
    - Head & Neck
    - Kidney
    - Liver
    - Lung
    - Ovarian
    - Pancreatic
    - Prostate
    - Skin
    - Stomach
    - Testis
    - Thyroid
    - Uterus
- Due to the high volume of data and memory access, further training would require more resources

Requirements

See pyproject.toml for the full list of dependencies and versions (Python ≥ 3.12). Install them with uv sync.

Libraries

- import torch
- import torch.nn as nn
- import torch.nn.functional as F
- import torchvision.transforms.functional as TF
- import segmentation_models_pytorch as smp
- import numpy as np
- import matplotlib.pyplot as plt

References:

1: The main manuscript

```latex
@article{gamper2020pannuke,
  title={PanNuke Dataset Extension, Insights and Baselines},
  author={Gamper, Jevgenij and Koohbanani, Navid Alemi and Graham, Simon and Jahanifar, Mostafa and Benet, Ksenija and Khurram, Syed Ali and Azam, Ayesha and Hewitt, Katherine and Rajpoot, Nasir},
  journal={arXiv preprint arXiv:2003.10778},
  year={2020}
}
```

2:

```latex
@inproceedings{gamper2019pannuke,
  title={Pannuke: An open pan-cancer histology dataset for nuclei instance segmentation and classification},
  author={Gamper, Jevgenij and Koohbanani, Navid Alemi and Benet, Ksenija and Khuram, Ali and Rajpoot, Nasir},
  booktitle={European Congress on Digital Pathology},
  pages={11--19},
  year={2019},
  organization={Springer}
}
```